

Song Recommendation Based on Acoustic Similarity Using Audio Feature Extraction and Clustering

Nishu Kumari

Shaily Mishra

*Department of Electronics and Communication Engineering Bharati
Vidyapeeth, Pune, India*

Abstract

Music recommendation systems usually depend on user listening history. This project builds a content-based recommender that uses only the sound of the songs. Each 30-second audio clip is converted into a vector of 70 acoustic features (MFCCs, chroma, spectral descriptors, zero-crossing rate, energy and tempo) with the `librosa` library. The features are standardised and reduced with Principal Component Analysis (PCA). Songs similar to a query song are found with cosine-distance nearest neighbours, and the collection is grouped with K-Means clustering, where the number of clusters is chosen by the silhouette score. The system was tested on the GTZAN dataset (999 usable clips, 10 genres). Using genre labels only for evaluation, the system reached a Precision@5 of 0.612 when PCA retained 99% of the variance, compared with about 0.10 for random recommendation. Lower PCA settings of 80% and 95% gave 0.559 and 0.603. K-Means with $k = 3$ separated calm (classical, jazz), rhythmic (pop, hip-hop, reggae) and loud (metal, rock) music without using labels. The system was also tested on 15 new songs that were not part of the dataset, which are shown as stars in the cluster plot.

Keywords: music recommendation, audio feature extraction, MFCC, cosine similarity, K-Means clustering, PCA

1. Introduction

Digital music services hold millions of songs, and no listener can browse such a catalogue by hand. Recommendation systems help by suggesting

songs a listener is likely to enjoy. Most of them use information such as what other users played, ratings and text tags [7]. These sources work well for popular songs, but a new or unpopular song has little or no listening history, so it cannot be recommended. This is called the cold-start problem. A content-based method avoids it by looking at the audio itself: if two songs have similar timbre, rhythm and energy, they can be recommended together even if nobody has played them yet.

Given a collection of audio files and one query song, the task of this project is to return the songs that sound most similar to the query. Each audio file is turned into a fixed-length vector of numbers (features). A similarity algorithm compares these vectors, and a clustering algorithm groups the collection so that its structure can be studied. The objectives are to extract acoustic features with a standard library, to recommend songs with cosine-distance nearest neighbours, to group songs with K-Means and choose the number of clusters objectively, to evaluate the result against a random baseline while testing three PCA settings (80%, 95% and 99% of variance retained), and to test the system on new songs that are not in the dataset.

The scope is limited to content-based recommendation on one public dataset plus a small set of new songs. User preferences, lyrics and large listening tests with people are outside the scope. The rest of the paper is organised as follows. Section 2 reviews related work, Section 3 describes the dataset, Section 4 explains the methodology, Section 5 gives the implementation, Section 6 presents the results, Section 7 discusses them, and Section 8 concludes.

2. Related Work

Recommender systems are commonly divided into collaborative filtering, which uses the behaviour of many users, and content-based filtering, which uses the properties of the items [8]. For music, the main challenges are very large catalogues, the effect of context and mood, and the cold-start problem [7].

Tzanetakis and Cook [1] showed that timbre, rhythm and pitch content can be used to classify music by genre, and they introduced the GTZAN collection used in this project. Timbre is often described with Mel-frequency cepstral coefficients (MFCCs) [4], and rhythm with the tempo found by beat tracking [9]. The `librosa` library [2] provides these and many other audio descriptors.

Principal Component Analysis (PCA) [10] reduces the number of features while keeping most of the variance. K-Means [11] with `k-means++` seeding [12] groups data into k clusters, and the silhouette score [5] can be used to choose k . Precision is a standard retrieval measure [13]. Sturm [6] reports faults in GTZAN, such as repeated clips and some doubtful labels, so results on this dataset should be read with care.

3. Dataset

The GTZAN dataset contains 1000 audio clips of 30 seconds each, in 10 genres with 100 clips per genre: blues, classical, country, disco, hip-hop, jazz, metal, pop, reggae and rock. The genre label of a clip is the name of its folder. One file in the jazz folder could not be read because it is damaged, and the code skipped it, so 999 clips were used (99 jazz and 100 of every other genre). The labels were not used to build the recommender or the clusters. They were used only afterwards to measure how well the recommendations and clusters match the genres. Because the genres are balanced, the chance of picking a song of the same genre by random guessing is about one in ten, which gives a baseline Precision@5 of about 0.10.

In addition to GTZAN, 15 new songs were downloaded from a free music site. They are full songs by independent artists, were never used to build the model, and were placed in folders named after their genre (five jazz, five metal and five pop). They are used only to test the finished system (Section 6.4).

4. Methodology

The system has five stages, shown in Figure 1. Audio files are converted to feature vectors, the features are standardised and reduced with PCA, and the result is used for nearest-neighbour recommendation and for K-Means clustering. The stages are run one after the other, and the output of each stage is the input of the next, so a change in an early stage, such as the choice of features, affects every later result. Feature extraction is the only stage that reads the audio files, and it is also the slowest, so its output is saved in a table and reused. The remaining stages work only on this table of numbers. The same standardisation and PCA that are fitted on the collection are also applied to any new song, so that a new song is placed in the same feature space as the songs already in the collection.

SONG RECOMMENDATION BASED ON ACOUSTIC SIMILARITY

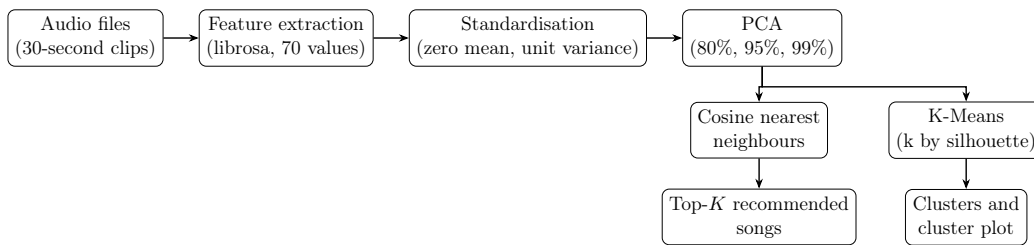


Figure 1: Overview of the recommendation pipeline.

Each clip was loaded as mono audio at 22,050 Hz with `librosa` [2], using at most the first 30 seconds. Every song was converted to a vector of 70 features (Table 1). Most descriptors are computed on short frames, which gives a sequence of values over time, and each sequence was summarised by its mean and, for several descriptors, its standard deviation. With the default settings of `librosa`, each frame has 2048 samples (about 93 ms) and the frames move forward by 512 samples (about 23 ms), so a 30-second clip gives about 1,300 frames for every descriptor. Taking the mean and the standard deviation turns this long sequence into two numbers, which gives every song a vector of the same length no matter how long its audio is. MFCCs describe timbre, such as the difference between a guitar and a violin [4]. Chroma values give the energy in each of the 12 pitch classes, which relates to harmony. The spectral centroid, bandwidth, rolloff and contrast describe how bright or noisy the sound is. The zero-crossing rate and RMS energy describe noisiness and loudness, and the tempo gives the speed in beats per minute. The tempo is a single value for the whole clip, found by beat tracking [9]. Together, these descriptors cover three main sides of music: timbre, harmony and rhythm with energy, so that two songs are close in the feature space only when they are alike in all of these.

The features have very different ranges (tempo is about 120, while many MFCC statistics are close to 0.1), so each feature j was standardised as $z_{ij} = (x_{ij} - \mu_j)/\sigma_j$, where μ_j and σ_j are its mean and standard deviation over all songs. PCA [10] was then applied to remove redundant information. The number of components was chosen automatically as the smallest number that keeps a target fraction of the variance: 80%, 95% or 99%.

For a query song with vector $\mathbf{a}$ in the PCA space, every other song $\mathbf{b}$ is

SONG RECOMMENDATION BASED ON ACOUSTIC SIMILARITY

Table 1: Features extracted from each song.

Feature group	Describes	Count
MFCC (20 coefficients, mean and std)	Timbre	40
Chroma (12 bins, mean)	Harmony and notes	12
Spectral centroid, bandwidth, rolloff (mean and std)	Brightness	6
Spectral contrast (7 bands, mean)	Peak-valley difference in the spectrum	7
Zero-crossing rate (mean and std)	Noisiness	2
RMS energy (mean and std)	Loudness	2
Tempo	Speed (beats per minute)	1
Total		70

compared by cosine similarity,

$$\text{sim}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}, \quad (1)$$

which compares the pattern of the features and not their size. The K most similar songs, excluding the query itself, are recommended. A brute-force search from `scikit-learn` [3] was used. A new song is processed in the same way, using the scaler and the PCA that were fitted on the 999 songs, and its recommendations are taken from the 999 songs.

K-Means [11] divides the songs into k clusters $C_1, \dots, C_k$ with centres μ_c by minimising

$$J = \sum_{c=1}^k \sum_{\mathbf{x} \in C_c} \|\mathbf{x} - \mu_c\|^2, \quad (2)$$

using `k-means++` seeding [12] and 10 random starts. No genre labels are used. The value of k was chosen from 2 to 15 by the mean silhouette score [5], where the silhouette of a song is $s(i) = (b(i) - a(i)) / \max\{a(i), b(i)\}$, with $a(i)$ the mean distance to its own cluster and $b(i)$ the mean distance to the nearest other cluster.

Recommendations were evaluated with $\text{Precision@}K$ for $K = 5$: for each query song, the fraction of its top- K recommendations with the same genre as the query, averaged over all N query songs,

$$\text{Precision@}K = \frac{1}{N} \sum_{i=1}^N \frac{|\{\text{top-}K \text{ songs of } i \text{ with the same genre as } i\}|}{K}. \quad (3)$$

With 10 equally sized genres, random recommendation gives about 0.10, which is used as the baseline. The complete procedure is given as pseudo code in Algorithm 1.

Algorithm 1 Song recommendation based on acoustic similarity

Require: Audio files, query song q , number of recommendations K

Ensure: K similar songs, song clusters, Precision@ K

```

1: for each audio file do
2:   Load the first 30 seconds of the song
3:   Compute MFCC, chroma, spectral features, ZCR, RMS and tempo
4:   Summarise them with mean and standard deviation  $\rightarrow$  vector of 70
   numbers
5: end for
6: Standardise every feature (mean 0, standard deviation 1)
7: Apply PCA, keeping 99% of the variance
8: Fit a nearest-neighbour model on the PCA data (cosine distance)
9: for  $k = 2$  to 15 do
10:   Run K-Means and compute the silhouette score
11: end for
12: Choose the  $k$  with the highest silhouette score and make the clusters
13: procedure RECOMMEND( $q, K$ )
14:   Get the PCA vector of  $q$  (a new song uses the same scaler and PCA)
15:   Compute the cosine similarity of  $q$  to every song in the collection
16:   return the  $K$  most similar songs, not including  $q$ 
17: end procedure
18: procedure EVALUATE( $K$ )
19:   for each song  $i$  do
20:     Take the  $K$  recommendations of  $i$  and count those with the same
     genre as  $i$ 
21:   end for
22:   return the average of (same-genre count /  $K$ ) over all songs
23: end procedure

```

5. Implementation and Experimental Setup

The system was written in Python 3.12 and run on a Windows laptop inside a virtual environment. The libraries used were **librosa** (audio features), **scikit-learn** (scaling, PCA, nearest neighbours, K-Means and the silhouette score), **pandas** and **numpy** (data handling), **joblib** (saving the model) and **matplotlib** (plots). The code is one script with numbered sections: it loads the feature table, scales and reduces the features, builds the similarity model, clusters the songs, evaluates them, draws the pictures, saves the

model, and finally tests the new songs. Feature extraction is slow, so it is done once and the table is saved. Table 2 lists the main settings, which were the same in all experiments except for the PCA variance fraction.

Table 2: Main parameter settings.

Parameter	Value
Sampling rate	22,050 Hz, mono
Clip length used	30 seconds
MFCC coefficients	20
Scaling	Standardisation (zero mean, unit variance)
PCA variance retained	80%, 95%, 99% (compared)
Similarity	Cosine distance, brute-force search
Recommendations per query (K)	5
Range of k tested for K-Means	2 to 15
K-Means random starts	10
Random seed	42

Three experiments were run. First, for each of the three PCA settings the recommender was built and its Precision@5 was computed over all 999 songs. Second, for the final setting (99%) the genre distribution of the clusters was printed, the cluster plot was saved, and an example recommendation was made for the blues clip `blues.00001`. Third, the 15 new songs were tested. For a new song, features were taken from a 30-second part that starts 30 seconds into the song, so that the intro is skipped and the segment is similar in length to the GTZAN clips.

6. Results

6.1. Effect of PCA on recommendation quality

Table 3 and Figure 2 compare the three PCA settings. Keeping more variance gave a higher Precision@5. The best result was 0.612 with 99% of the variance (56 components), a relative gain of about 9.5% over 80% (14 components) and about 1.5% over 95% (38 components). All three settings are far above the random baseline of about 0.10, and the best setting is about six times better than random.

The best silhouette score of K-Means is highest for the smallest PCA setting (0.233 at 80%) and falls as more components are kept (0.190 and

SONG RECOMMENDATION BASED ON ACOUSTIC SIMILARITY

Table 3: Precision@5 for different PCA settings (999 songs).

Variance kept	Components	Best k (silhouette)	Precision@5
80%	14	3 (0.233)	0.559
95%	38	3 (0.190)	0.603
99%	56	3 (0.181)	0.612

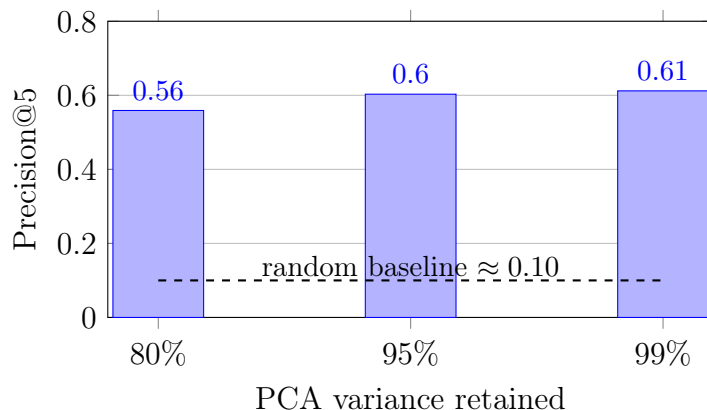


Figure 2: Precision@5 for the three PCA settings, with the random baseline.

0.181). The silhouette score is computed from Euclidean distances in the reduced space, and it is not the same quantity as Precision@5, which uses cosine similarity, so the two do not have to agree. The choice of 99% was made on the recommendation score, since recommendation is the main goal of the project.

6.2. Example recommendation

Table 4 shows the top five recommendations for the blues clip `blues.00001` with the 99% setting. Only one of the five songs is blues, and the others are reggae and rock, so the precision for this query is 0.2, which is below the average of 0.612. The similarity values are between 0.54 and 0.65, so no song is a very close match. A single query is only an example. The score over all songs is the more reliable measure, because genres differ in how easy they are to separate.

SONG RECOMMENDATION BASED ON ACOUSTIC SIMILARITY

Table 4: Top-5 recommendations for `blues.00001` (99% variance).

Rank	Song	Genre	Similarity
1	rock.00014	rock	0.648
2	reggae.00025	reggae	0.590
3	blues.00092	blues	0.576
4	reggae.00036	reggae	0.555
5	reggae.00009	reggae	0.543

6.3. Clustering

With 99% of the variance kept, the silhouette score selected $k = 3$ (score 0.181). Table 5 shows how the clips of each genre were distributed over the three clusters.

Table 5: Number of clips of each genre in each K-Means cluster ($k = 3$, 99% variance).

Genre	Cluster 0	Cluster 1	Cluster 2	Total
Blues	53	3	44	100
Classical	95	3	2	100
Country	59	23	18	100
Disco	4	48	48	100
Hip-hop	0	73	27	100
Jazz	70	12	17	99
Metal	0	2	98	100
Pop	7	93	0	100
Reggae	18	75	7	100
Rock	19	32	49	100
Total	325	364	310	999

The clusters follow a clear acoustic pattern even though no labels were used. Cluster 0 is mainly calm music (classical 95, jazz 70, country 59, blues 53), cluster 1 is mainly rhythmic music (pop 93, reggae 75, hip-hop 73, disco 48), and cluster 2 is mainly loud music (metal 98, rock 49, disco 48, blues 44). About 95% of the classical clips are in cluster 0, 98% of the metal clips are in cluster 2, and 93% of the pop clips are in cluster 1. Other genres are split: disco is divided equally between clusters 1 and 2, blues between

clusters 0 and 2, and rock is spread over all three. With 10 genres and only 3 clusters, a cluster cannot match a single genre. The useful result is that genres with a similar sound share a cluster. Figure 3 shows the clusters and the true genres in a two-dimensional PCA projection. The clusters touch and overlap, which agrees with the low silhouette score.

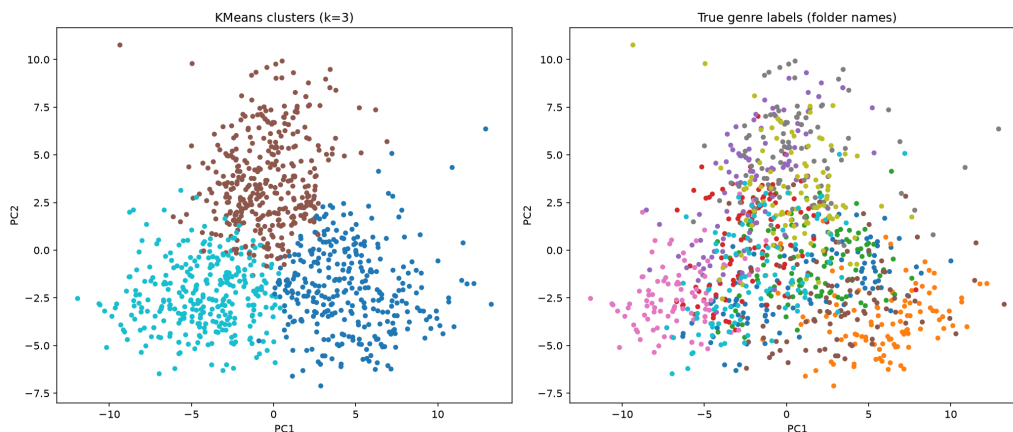


Figure 3: Left: K-Means clusters ($k = 3$). Right: true genre labels. Both are shown on the first two principal components.

6.4. Testing on new songs

To check the system on songs it had never seen, 15 new songs (five jazz, five metal and five pop) were processed with the same scaler and PCA as the 999 songs. Each new song was assigned to a cluster, and its five most similar songs were taken from the 999 songs. The Precision@5 on the 15 new songs is **0.XXX**, compared with 0.612 for the 999 songs and about 0.10 for random recommendation.

Figure 4 shows the new songs as numbered stars on top of the 999 songs. A star is a different shape from the dots so that it can be found easily. On the left, the colour of each star is the cluster that the system assigned to it. On the right, the colour of each star is the true genre of the song, so a star that lies among the dots of its own colour was placed well by the system.

This test is small and has limits. With only 15 songs, a single song can change the average Precision@5 by up to 0.067, so the number is only a rough check. The new songs are full songs by independent artists, while GTZAN

SONG RECOMMENDATION BASED ON ACOUSTIC SIMILARITY

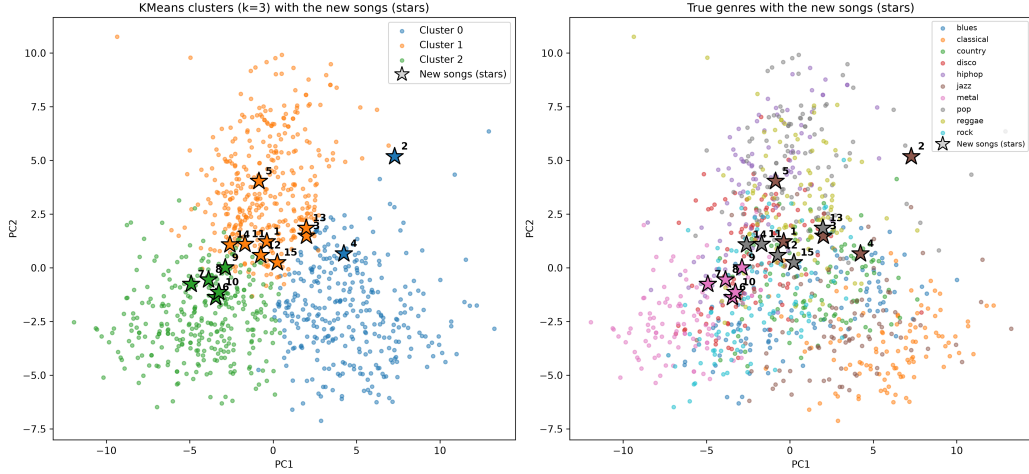


Figure 4: The 15 new songs (stars) on the cluster plot of the 999 songs. Left: coloured by cluster. Right: coloured by true genre.

contains 30-second clips, so their recording style can differ from the training collection, and the new songs cover only three genres.

7. Discussion

7.1. Interpretation of the results

A Precision@5 of 0.612 means that, on average, about three of the five recommended songs share the genre of the query. This is far better than the random baseline of about 0.10, so the 70 acoustic features and the cosine similarity capture real information about how songs sound. The errors are not random. In the example and in the cluster table, blues, rock and reggae appear together. These genres often use guitar-based instrumentation and a moderate tempo, so their summary features are close. The clustering supports this view, since it separates calm, rhythmic and loud songs but cannot separate genres inside each group.

7.2. Design choices and measurement

Standardisation was used because the features have different units and ranges, cosine similarity because it compares the pattern of the features and not their size, and PCA to remove redundancy. The PCA experiment shows that how much information is kept matters: a very small representation

(80%) lost detail that helps recommendation. The effect of standardisation and of cosine versus Euclidean distance was not tested separately, so these choices are supported by common practice and not by an experiment here. Using the genre label as the definition of relevance is simple and objective, but it is only an approximation of what a listener wants. A recommended song with a different genre label is counted as an error even if a listener might like it, so the reported precision is a lower bound on usefulness, and a listening test would give a more direct measure. Feature extraction is the slowest step, but it is done once and saved. The implementation ranks all songs for every query, which is acceptable for 999 songs, and a catalogue with millions of songs would need an approximate nearest-neighbour index.

7.3. Limitations

- The system is content-based and does not use listening history or user preferences, so it cannot adapt to a single listener.
- Only 30 seconds of each song are analysed, and only summary statistics (mean and standard deviation) are kept, so changes over time within a song are lost.
- Acoustically similar songs do not always share a genre, and GTZAN has known faults such as repeated clips and some doubtful labels [6].
- The results come from one dataset and one run with a fixed random seed, and the new-song test uses only 15 songs from three genres.
- The number of clusters chosen by the silhouette score is small ($k = 3$), so the clusters describe broad styles and not individual genres.

8. Conclusion and Future Work

This project built a content-based song recommender that uses only audio. Each song was described by 70 acoustic features, the features were standardised and reduced with PCA, similar songs were found with cosine nearest neighbours, and the collection was grouped with K-Means. On the GTZAN dataset the system reached a Precision@5 of 0.612, about six times better than random recommendation, and it improved as more PCA variance was kept (0.559, 0.603 and 0.612 for 80%, 95% and 99%). K-Means with $k = 3$ found a meaningful structure without labels, separating calm,

rhythmic and loud music. The errors mostly come from genres that sound alike, such as blues, rock and reggae. A test on 15 new songs, shown as stars on the cluster plot, confirmed that the system can be applied to songs outside the dataset.

Several extensions are possible. Richer features, or embeddings from a pretrained audio model, could separate similar genres better. Other distance measures and clustering methods could be compared, and the effect of standardisation could be tested directly. The system could be evaluated on more datasets, on more new songs and with a listening test. Finally, combining the audio-based similarity with user listening history would give a hybrid recommender that keeps the advantage of working for new songs.

Appendix A. How to run the system

The code is run with `python final_code.py` after installing the libraries with `pip install librosa scikit-learn pandas matplotlib joblib soundfile`. The songs of the dataset are expected in a folder `genres_original`, and the new songs in genre folders inside `new_songs`. The script creates `features.csv`, the saved model and a `results` folder with the pictures and tables.

References

- [1] G. Tzanetakis, P. Cook, Musical genre classification of audio signals, *IEEE Transactions on Speech and Audio Processing* 10 (5) (2002) 293–302.
- [2] B. McFee, C. Raffel, D. Liang, D. P. W. Ellis, M. McVicar, E. Battenberg, O. Nieto, librosa: Audio and music signal analysis in Python, in: *Proceedings of the 14th Python in Science Conference*, 2015, pp. 18–25.
- [3] F. Pedregosa, et al., Scikit-learn: Machine learning in Python, *Journal of Machine Learning Research* 12 (2011) 2825–2830.
- [4] S. Davis, P. Mermelstein, Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences, *IEEE Transactions on Acoustics, Speech, and Signal Processing* 28 (4) (1980) 357–366.

- [5] P. J. Rousseeuw, Silhouettes: A graphical aid to the interpretation and validation of cluster analysis, *Journal of Computational and Applied Mathematics* 20 (1987) 53–65.
- [6] B. L. Sturm, The state of the art ten years after a state of the art: Future research in music information retrieval, *Journal of New Music Research* 43 (2) (2014) 147–172.
- [7] M. Schedl, H. Zamani, C.-W. Chen, Y. Deldjoo, M. Elahi, Current challenges and visions in music recommender systems research, *International Journal of Multimedia Information Retrieval* 7 (2) (2018) 95–116.
- [8] F. Ricci, L. Rokach, B. Shapira, P. B. Kantor (Eds.), *Recommender Systems Handbook*, Springer, 2011.
- [9] D. P. W. Ellis, Beat tracking by dynamic programming, *Journal of New Music Research* 36 (1) (2007) 51–60.
- [10] I. T. Jolliffe, J. Cadima, Principal component analysis: a review and recent developments, *Philosophical Transactions of the Royal Society A* 374 (2065) (2016) 20150202.
- [11] S. P. Lloyd, Least squares quantization in PCM, *IEEE Transactions on Information Theory* 28 (2) (1982) 129–137.
- [12] D. Arthur, S. Vassilvitskii, k-means++: The advantages of careful seeding, in: *Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms*, 2007, pp. 1027–1035.
- [13] C. D. Manning, P. Raghavan, H. Schütze, *Introduction to Information Retrieval*, Cambridge University Press, 2008.

Appendix B. Source Code and Reproducibility

Appendix B.1. Code availability

The complete implementation of the proposed system, together with the extracted feature table, the saved model, the result figures and the documentation, is publicly available in the following GitHub repository:

<https://github.com/nishukri/song-recommendation-acoustic-similarity>
github.com/nishukri/song-recommendation-acoustic-similarity

The repository is public and can be opened without an account. All results reported in Section 6 were produced with the code in this repository, using a fixed random seed of 42 for PCA and K-Means, so the same numbers are obtained when the code is run again on the same feature table.

Appendix B.2. Repository contents

Table B.6 lists the main files and folders of the repository and what each of them contains.

Table B.6: Contents of the GitHub repository.

File or folder	Description
<code>music_recommender.py</code>	Main script: feature loading, standardisation, PCA, cosine nearest neighbours, K-Means, evaluation, figures and the test on new songs
<code>features.csv</code>	Table of the 70 acoustic features of the 999 GTZAN clips, with the path and genre of each clip
<code>music_model.pkl</code>	Saved model (scaler, PCA, nearest-neighbour model and K-Means)
<code>results/</code>	Figures and tables produced by the script, including the PCA comparison, the precision for each genre and the results on the new songs
<code>clusters.png</code>	Cluster plot of the 999 songs
<code>clusters_with_new_songs.png</code>	Cluster plot with the 15 new songs shown as stars
<code>test_new_songs.py</code>	Script used to test songs that are not part of the dataset
<code>unseen_results.csv</code>	Recommendations and cluster labels for the unseen songs
<code>output_final.txt</code>	Complete printed output of the final run
<code>requirements.txt</code>	List of the Python libraries needed
<code>README.md</code>	Short description of the project and instructions to run it

Appendix B.3. Software environment

The experiments were run in Python 3.12 inside a virtual environment on a Windows laptop. The main libraries were `librosa` for audio features, `scikit-learn` for scaling, PCA, nearest neighbours, K-Means and the silhouette score, `pandas` and `numpy` for data handling, `joblib` for saving the model, and `matplotlib` for the figures. A newer version of Python (3.14) was not used, because `librosa` could not be installed with it at the time of the experiments. The libraries can be installed with one command from the file `requirements.txt`.

Appendix B.4. Steps to reproduce the results

The results can be reproduced in four steps.

1. Download the repository and install the libraries with `pip install -r requirements.txt`.
2. Run `python music_recommender.py`. Because `features.csv` is included, the slow feature extraction step is skipped, and the script prints the evaluation, the PCA comparison, the example recommendation and the cluster table, and saves the figures in the `results` folder.
3. To extract the features again from the audio, download the GTZAN dataset, place it in a folder named `genres_original`, delete `features.csv` and run the script again. This step takes about 15 to 30 minutes on an ordinary laptop.
4. To test new songs, place them in genre folders inside a folder named `new_songs` (for example `new_songs/jazz`) and run the script again. A new song is processed with the same scaler and PCA as the 999 songs.

Appendix B.5. Data availability and limitations

The GTZAN dataset is not included in the repository because of its size (about 1.2 GB). It can be downloaded separately from Kaggle and placed in the folder `genres_original`, as described in Appendix A. The 15 new songs are also not included, because they are full-length recordings whose redistribution may not be allowed, so the repository contains only their extracted features and results. The reported numbers may differ slightly if a different version of `librosa` or `scikit-learn` is used, because small changes in the feature extraction or in the clustering algorithm can change the last decimal places.